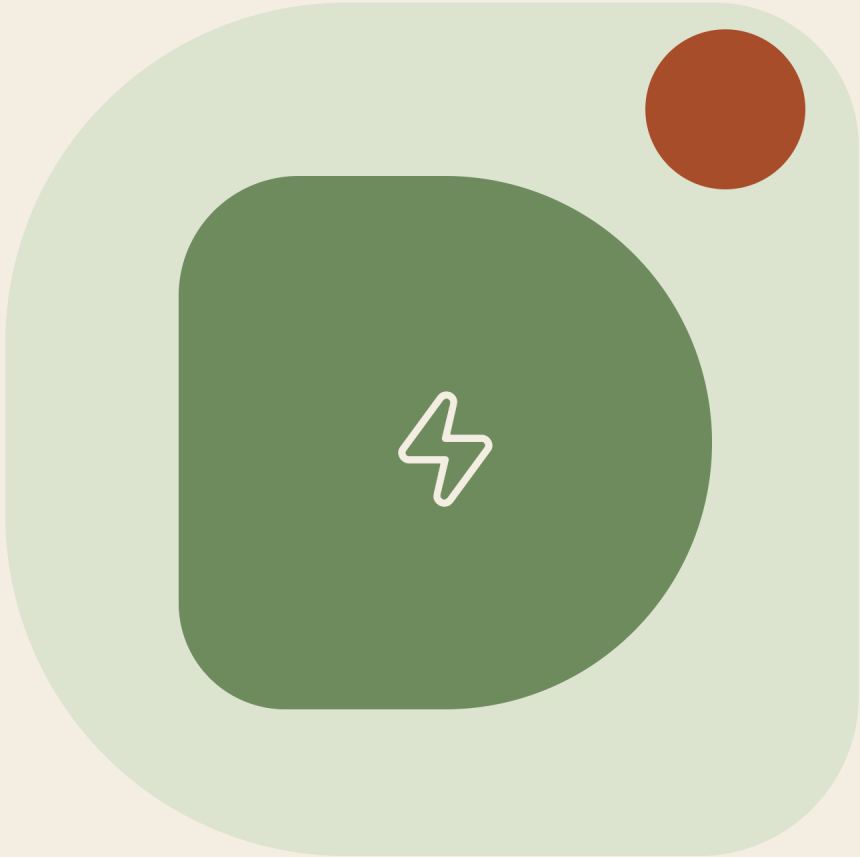
SHARING KNOWLEDGE · SOFTWARE TESTING

Performance & Load Testing

Dukhaan Kamimpangan



INTERAKTIF

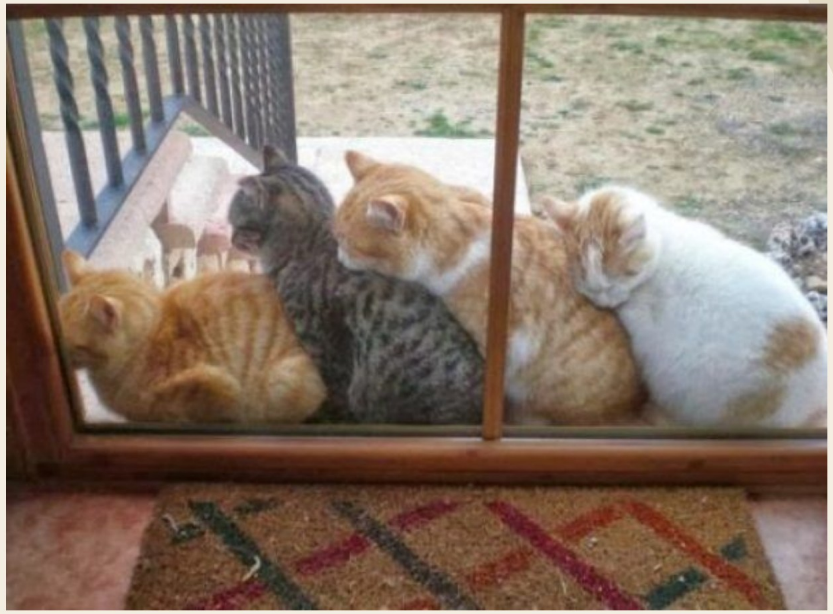
Sebelum Kita Mulai...

Pernah nggak aplikasi instagram kalian tiba-tiba lemot pas baru dibuka?



INTERAKTIF

Hal tersebut diakibatkan



Banyak user yang secara bersamaan membuka aplikasi tersebut

sehingga mengakibatkan antrean dari server ke client



SEBELUM MULAI

Disclaimer juga

Cuking family is back! Mereka bakal bantu jelasin kenapa server bisa ngos-ngosan, dan mas mbak yang keren juga akan bantu buat experiment nanti.



01



Aplikasi Cepat buat Apa?

Jalan di laptop sendiri belum tentu kuat saat dipakai ribuan orang.

01 · KENAPA PERFORMA

“It Works on My Machine”

DEVELOP

1 user: kamu sendiri. Semua terasa ngebut.

→

DEMO

5 user: dosen dan teman sekelompok. Masih aman.

→

LAUNCH

1.000 user di jam yang sama. Mulai terasa berat.

→

DOWN

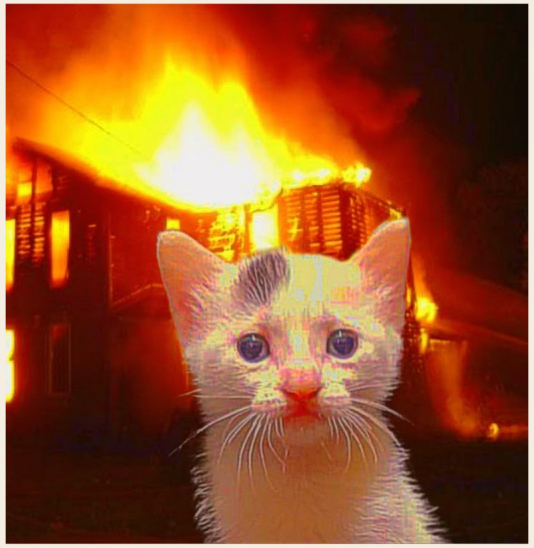
Timeout, error 500, user kabur.

DEVELOP · 1 user

respon: 80 ms

SERVER

CPU 0%



FUN FACT

1 jam

waktu yang dibutuhkan OpenAI nambah **1 juta user baru** waktu ada trend foto Ghibli (Maret 2025). Waktu pertama rilis butuh 5 hari. CEO-nya sampai bilang: **“our GPUs are melting”**.



ANOTHER NEWS

Portal pendaftaran CPNS (SSCASN) hampir tiap tahun susah diakses karena diserbu bareng. Tahun 2023, batas pendaftaran sampai **diperpanjang** dari 9 ke 11 Oktober.

Sumber: Sam Altman via Windows Central, 2025 · CNBC Indonesia, 2023

01 · KENAPA PERFORMA

Momen yang Sering Bikin Server Tumbang

War Tiket

Ribuan orang klik “beli” di detik yang sama.

Flash Sale

Promo jam 12.12 tepat, traffic melonjak puluhan kali.

Isi FRS

Satu angkatan login bareng begitu jadwal dibuka.

Pengumuman

Hasil seleksi rilis jam 15.00, semua refresh berkali-kali.









01 · KUIS BENTAR

Aman atau Tidak?

Skenario: aplikasi kalian lolos **semua unit test dan functional test**. Besok mau dipakai **5.000 mahasiswa** sekaligus untuk pendaftaran. Aman?

Aman

Aman't

Cuman diatas yang tau



01 · JAWABAN KUIS

Jawaban: Belum Tentu

TEPAT!



Functional test cuman ngecek “benar atau salah”, bukan “kuat atau tidak”

YANG SUDAH DITES

Login berhasil, data tersimpan, hitungan benar. Semuanya dengan **1 user**.

YANG BELUM DITES

Berapa lama respons saat 5.000 user? Kapan mulai error? Database kuat atau tidak?

02



Fundamental Performance Testing

Apa yang diukur, cara membacanya, dan di mana biasanya macet.

Functional vs Non-Functional Testing

FUNCTIONAL

“Apakah fiturnya bekerja dengan benar?”

Unit test, integration test, end-to-end test. Inputnya A, hasilnya harus B.

NON-FUNCTIONAL

“Seberapa cepat, kuat, dan stabil?”

Performance, load, security, usability. Hari ini fokus di **performance & load**.





Sesi lalu kita bahas **security**. Sekarang saudaranya: aplikasi yang aman tapi tumbang tiap rame tetap gagal di sisi **Availability**.

02 · FUNDAMENTAL

Anggap Aplikasi Itu Warung Bakso

Di warung bakso	Di aplikasi
Pelanggan yang datang	Virtual User (VU)
Pesanan	Request
Lama nunggu bakso datang	Response time
Mangkok keluar per menit	Throughput
Pesanan salah atau batal	Error rate
Kompor cuma dua	Resource (CPU, RAM, koneksi DB)



02 · FUNDAMENTAL

Metrik yang Wajib Di identifikasi

Response Time

Waktu dari request dikirim sampai respons diterima. Satuan ms.

Throughput

Berapa request yang sanggup dilayani per detik (RPS).

Error Rate

Persentase request gagal: 5xx, timeout, koneksi ditolak.

Concurrent Users

Berapa user yang aktif bersamaan. Di k6 disebut VU.

02 · FUNDAMENTAL

Kenapa Rata-Rata Itu Menipu



Ilustrasi: 100 request masuk satu per satu. 90 selesai dalam 100 ms, 10 butuh 3.000 ms.

Rata-rata **390 ms** terlihat oke.

Tapi tidak ada satu user pun yang ngalamin 390 ms. Yang ada: kebanyakan cepat, dan **1 dari 10 user nunggu 3 detik**.

Makanya di dunia performance kita pakai **persentil**.

02 · FUNDAMENTAL

Persentil: p50, p90, p95, p99

p95 = 500 ms artinya 95% request selesai dalam 500 ms atau kurang, dan 5% sisanya lebih lambat dari itu.

p50

Median. Pengalaman user “biasa”.

p90

9 dari 10 user sudah dapat respons.

p95

Patokan paling umum untuk target SLA.

p99

Ekor paling lambat. Sering disebut “tail latency”.

Di contoh tadi: rata-rata 390 ms, p50 100 ms, tapi **p95 3.000 ms**. Ketahuan kan?

02 · FUNDAMENTAL

Persentil dalam 100 Orang

100 user diurutkan dari yang paling cepat dapat respons

user ke-4 · 46 ms



p50 · 120 ms

User ke-50. Separuh user dapat respons lebih cepat dari ini.

p90 · 300 ms

User ke-90. Tinggal 10 user yang lebih lambat.

p95 · 500 ms

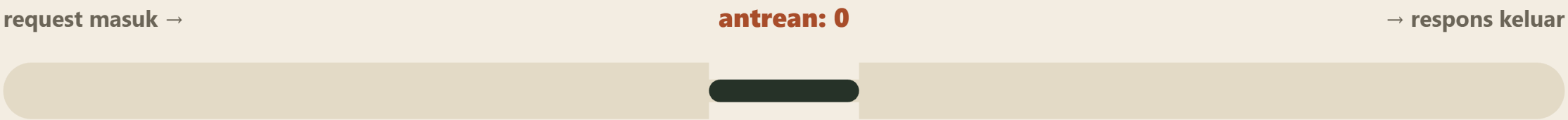
User ke-95. Angka inilah yang dicek threshold p(95)<500.

p99 · 1.800 ms

User ke-99. Cuma 1 yang lebih lambat, tapi dia yang paling kesal.

02 · FUNDAMENTAL

Di Mana Biasanya Macetnya?



JARINGAN

Payload terlalu besar, server jauh.

APLIKASI

Loop berat, proses sinkron yang blocking, tanpa cache.

DATABASE

Query N+1, tanpa index, connection pool habis.

API LUAR

Payment gateway atau layanan pihak ketiga yang lambat.

SERVER

CPU 100%, RAM penuh, disk I/O lambat.

Bagian paling sempit menahan semua yang lain. Di aplikasi CRUD ala tugas kuliah, tersangka nomor satu hampir selalu **database**.

03



Jenis-Jenis Performance Test

Beda pertanyaan, beda bentuk beban. Pilih test dari pertanyaannya.

03 · JENIS TEST

Jenis jenis Performance testing

Smoke

“Script dan sistemnya jalan nggak?”

Average Load

“Kuat di traffic normal sehari-hari?”

Stress

“Kuat kalau traffic di atas normal?”

Spike

“Kuat saat lonjakan mendadak?”

Soak

“Tetap stabil kalau jalan berjam-jam?”

Breakpoint

“Batas maksimalnya di mana?”

Smoke Test



Jumlah VU ↑ · Waktu → · simulasi 1 menit



> **DEFINISI**
>
> Test dengan beban sangat kecil untuk memastikan script dan sistem berfungsi sebelum test yang lebih berat.

TUJUAN

Memastikan script benar dan sistem merespons wajar dengan beban minimal.

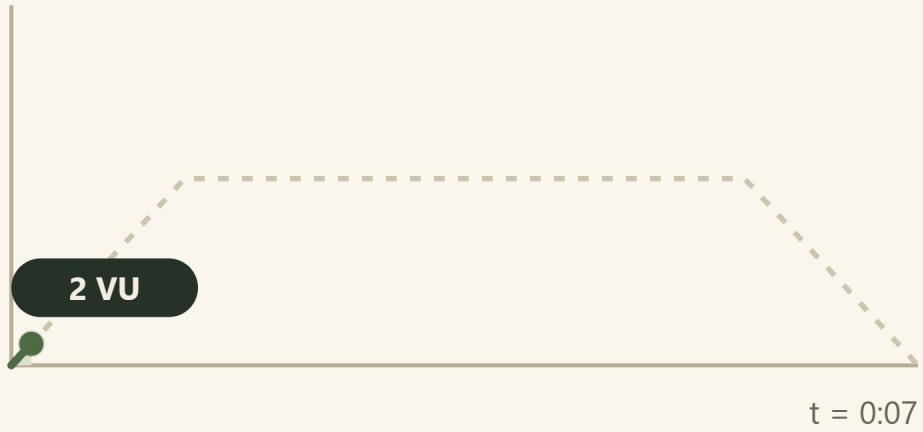
PROFIL BEBAN

1–5 VU, sekitar 1 menit.

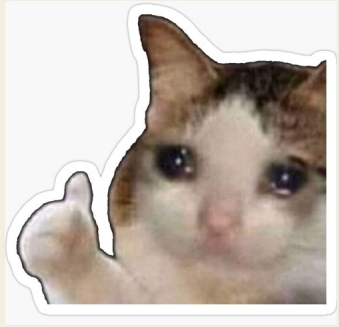
KAPAN

Selalu jadi langkah pertama, dan tiap kali script atau kode berubah.

Average Load Test



Ramp-up · tahan · ramp-down · simulasi 5 menit



DEFINISI

Test yang mensimulasikan jumlah user normal sehari-hari untuk mengukur performa di kondisi biasa.

TUJUAN

Mengukur performa di traffic normal yang diperkirakan.

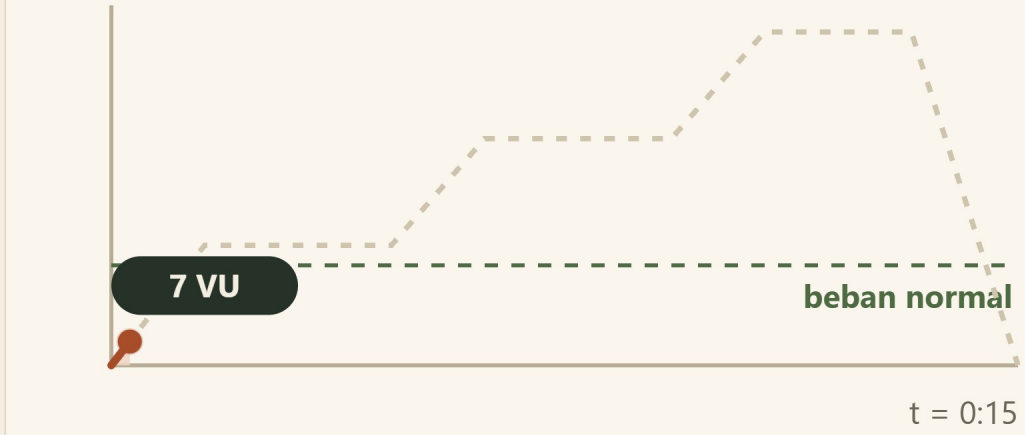
PROFIL BEBAN

Naik pelan ke jumlah user normal, ditahan beberapa menit, lalu turun.

KAPAN

Sebelum rilis, untuk dapat baseline yang dibandingkan antar versi.

Stress Test



Naik bertingkat melewati beban normal · simulasi 12 menit



DEFINISI

Test dengan beban di atas normal untuk melihat cara sistem bertahan, melambat, atau gagal.

TUJUAN

Melihat perilaku sistem saat beban di atas normal: melambat wajar atau langsung ambruk?

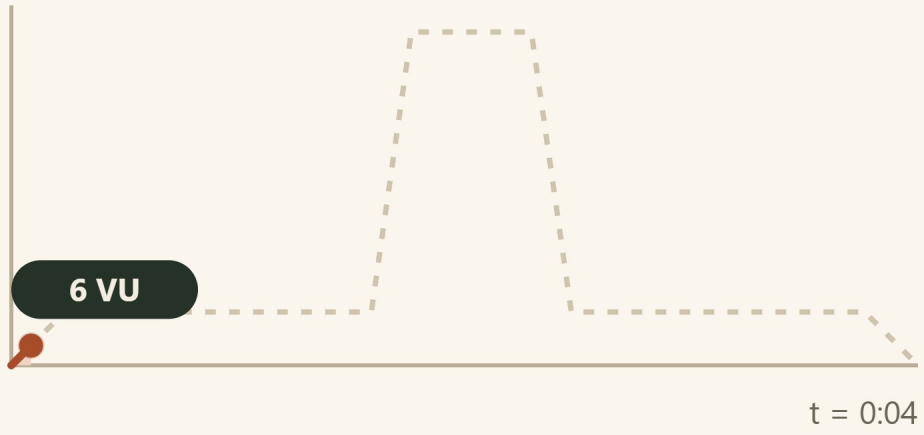
PROFIL BEBAN

Naik bertingkat, misalnya 150%–200% dari beban normal.

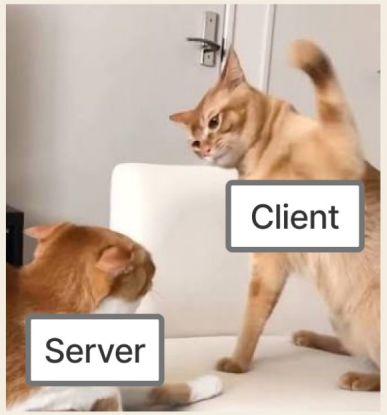
KAPAN

Menjelang event besar: promo, pendaftaran, ujian online.

Spike Test



Lonjakan mendadak lalu turun lagi · simulasi 3 menit



DEFINISI

Test dengan lonjakan user yang tiba-tiba dan singkat, lalu turun lagi ke normal.

TUJUAN

Mengecek apakah sistem tahan lonjakan tiba-tiba dan bisa pulih setelahnya.

PROFIL BEBAN

Dari sedikit ke sangat banyak dalam hitungan detik, tanpa ramp-up.

KAPAN

War tiket, flash sale, pengumuman jam tertentu, link viral.

Soak Test (Endurance)



Beban datar berjam-jam · RAM pelan-pelan naik (memory leak)



DEFINISI

Test dengan beban normal yang dijalankan sangat lama untuk menangkap masalah yang muncul seiring waktu.

TUJUAN

Menemukan masalah yang baru muncul setelah lama: memory leak, log memenuhi disk, koneksi tidak ditutup.

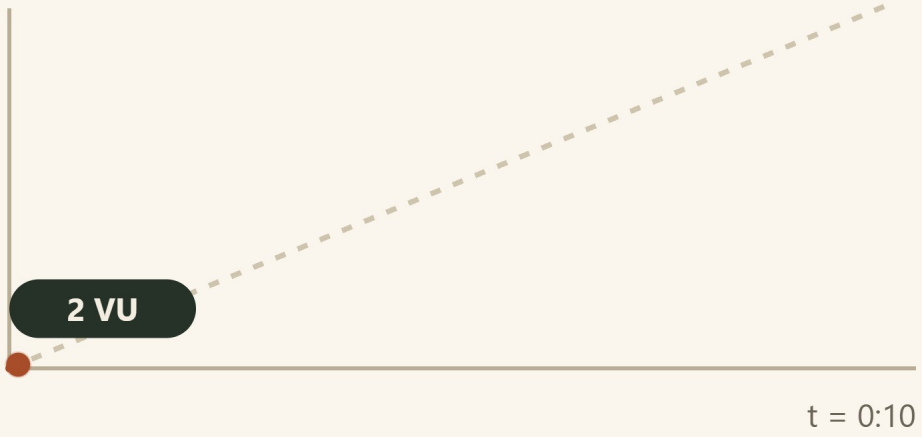
PROFIL BEBAN

Beban normal, ditahan beberapa jam sampai semalaman.

KAPAN

Aplikasi yang jalan 24/7. Tidak kita praktikkan hari ini karena butuh waktu lama.

Breakpoint Test



Naik terus sampai sistem gagal · di situlah kapasitas maksimalnya



DEFINISI

Test dengan beban yang terus dinaikkan sampai sistem gagal, untuk menemukan batas kapasitasnya.

TUJUAN

Mencari kapasitas maksimal: di angka berapa sistem mulai gagal.

PROFIL BEBAN

Naik terus tanpa henti, test dihentikan begitu error melonjak.

KAPAN

Perencanaan kapasitas: “server ini cukup untuk berapa user?”

03 · KUIS BENTAR

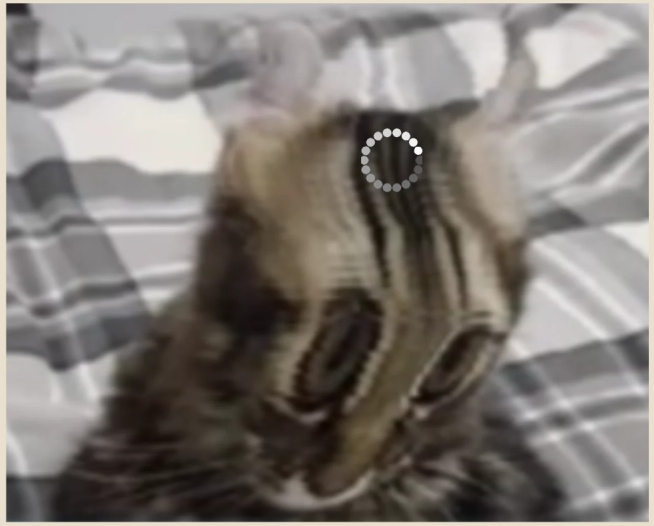
Tebak: Ini Termasuk Test Apa?

Skenario: pengumuman hasil seleksi dibuka **jam 15.00 tepat**. Dalam 1 menit, jumlah user naik dari 200 jadi **4.000**. Tim ingin tahu apakah server bertahan.

Load Test

Soak Test

Spike Test



03 · JAWABAN KUIS

TEPAT!

Jawaban: Spike Test

Lonjakan 20× dalam 1 menit = spike

Bukan load test (itu traffic normal), bukan soak test (itu soal durasi panjang). Kata kuncinya **mendadak**: user datang serentak di satu jam yang sama, tanpa ramp-up.



03 · KUIS BENTAR

Satu Lagi, Test Apa Ini?

Skenario: aplikasi lancar saat dites 10 menit. Tapi setelah jalan **6 jam** dengan traffic biasa, pemakaian RAM terus naik lalu server **restart sendiri**.

Stress Test

Soak Test

Breakpoint Test



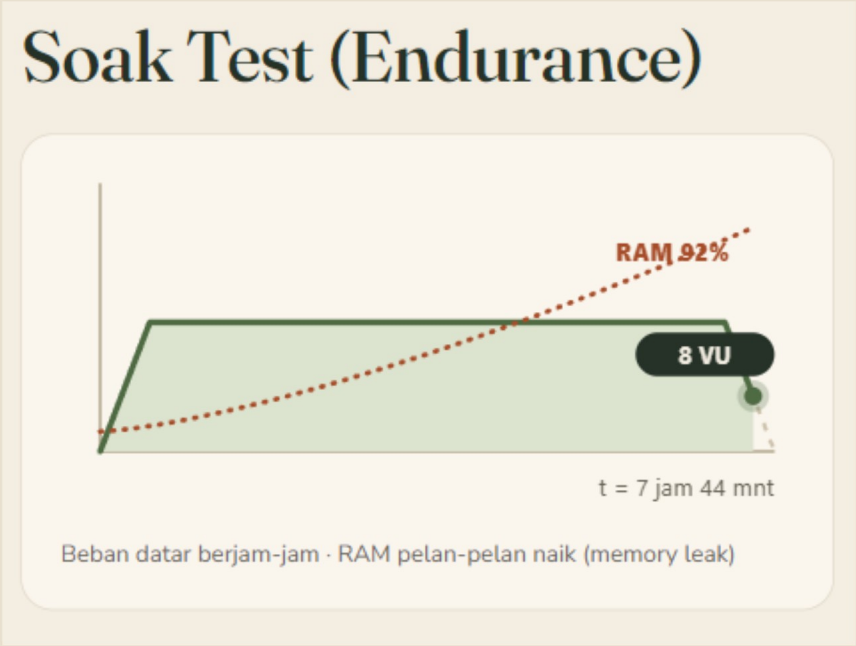
03 · JAWABAN KUIS

TEPAT!

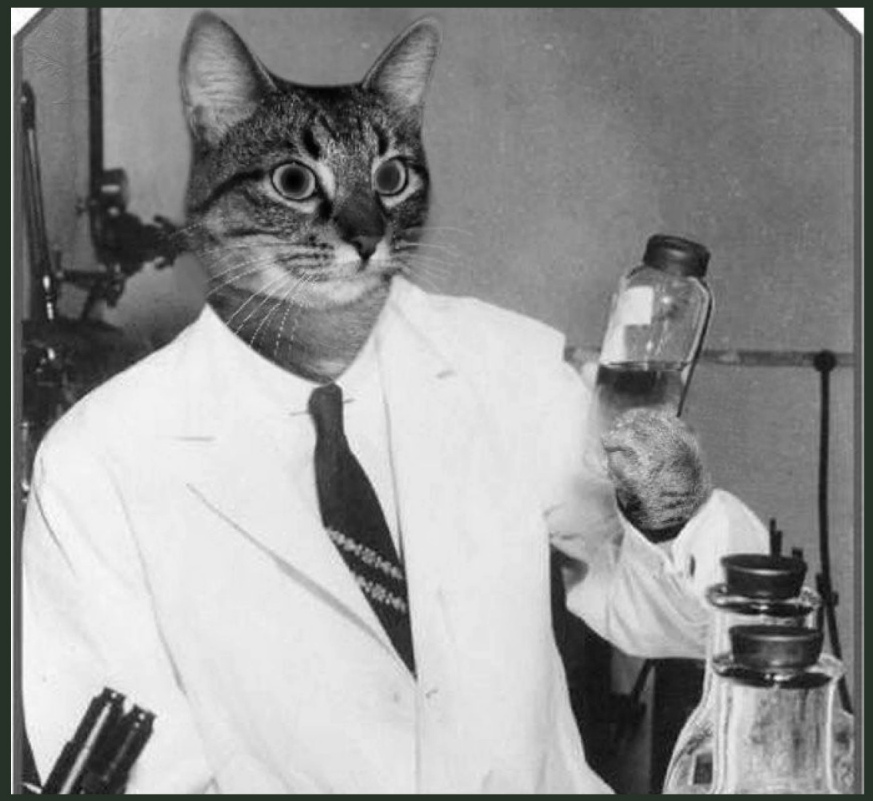
Jawaban: Soak Test

Traffic biasa + waktu lama + RAM terus naik = memory leak

Bebannya tidak di atas normal, jadi bukan stress atau breakpoint. Masalahnya baru kelihatan karena **durasi**. Test 10 menit tidak akan pernah menangkap ini.



04



Tools Performance Test

Banyak Tool, Pilih yang Cocok

Tool	Script ditulis pakai	Cara pakai	Biasanya dipilih oleh
Apache JMeter	GUI, disimpan sebagai XML	Klik di GUI	Tim QA yang suka GUI
Locust	Python	Kode + web UI	Tim Python
Grafana k6	JavaScript / TypeScript	Kode + CLI	Tim Developer

04 · K6

Kenapa k6?

Script = JavaScript

Test ditulis seperti kode biasa, bisa di-commit ke repo.

Ringan

Satu program CLI, cukup jalan dari laptop kalian.

Threshold bawaan

Hasil langsung lulus atau gagal, cocok untuk CI.

Open source

Dikelola Grafana Labs. Versi 1.0 rilis Mei 2025, sekarang seri 2.x.



Anatomi Script k6: Empat Tahap

1 · INIT

`import, options`

Kode di luar function. Menyiapkan konfigurasi.

→

2 · SETUP

`setup()`

Jalan sekali di awal. Contoh: login, ambil token.

→

3 · VU CODE

`default function`

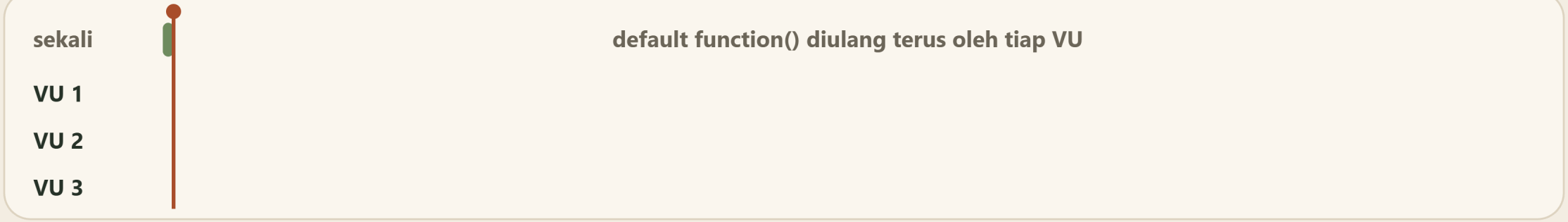
Diulang terus oleh tiap VU. Inti skenario.

→

4 · TEARDOWN

`teardown()`

Jalan sekali di akhir. Bersih-bersih data test.



Script Pertama: 12 Baris Saja

```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  vus: 10,
  duration: '30s',
};

export default function () {
  http.get('http://localhost:3000/api/products');
  sleep(1);
}
```

VUS: 10

10 user palsu jalan bersamaan.

DURATION

Test berlangsung 30 detik.

SLEEP(1)

Think time 1 detik, biar mirip user asli.

```
k6 run script1.js
```

Check: Bukan Cuma Cepat, Tapi Benar

```
import { check, sleep } from 'k6';

export default function () {
  const res = http.get(`${BASE}/api/products`);

  check(res, {
    'status 200': (r) => r.status === 200,
    'di bawah 500ms': (r) =>
      r.timings.duration < 500,
  });
  sleep(1);
}
```

Server cepat membalas **error 500** itu bukan prestasi. Check memastikan responsnya juga **benar**.

HATI-HATI

Check yang gagal **tidak** menggagalkan test. Ia cuma dicatat sebagai persentase. Untuk gagal beneran, pakai threshold.

Stages: Menggambar Bentuk Beban

```
export const options = {
  stages: [
    { duration: '1m', target: 20 },
    { duration: '3m', target: 20 },
    { duration: '1m', target: 0 },
  ],
};
```



Ganti angka di **stages**, maka load test berubah jadi stress, spike, atau breakpoint. Script yang sama, bentuk beban yang beda.

Threshold: SLO yang Bisa Lulus atau Gagal

```
export const options = {
  thresholds: {
    // 95% request di bawah 500 ms
    http_req_duration: ['p(95)<500'],
    // error di bawah 1%
    http_req_failed: ['rate<0.01'],
    // 99% check lolos
    checks: ['rate>0.99'],
  },
};
```

SEMUA TERPENUHI

Test lulus, exit code 0.

SATU SAJA DILANGGAR

Test gagal, exit code bukan 0. Di CI, pipeline langsung merah.

Endpoint yang Butuh Login

```
const BASE = __ENV.BASE_URL;
export function setup() {
  const res = http.post(`${BASE}/api/auth/login`,
    JSON.stringify({
      email: __ENV.TEST_EMAIL,
      password: __ENV.TEST_PASSWORD,
    }),
    { headers: { 'Content-Type': 'application/json' } });
  return { token: res.json('accessToken') };
}
export default function (data) {
  http.get(`${BASE}/api/products`, {
    headers: { Authorization: `Bearer ${data.token}` },
  });
}
```

Login **sekali** di setup(), token dibagikan ke semua VU lewat parameter **data**.

INGAT GITLEAKS!

Email dan password test lewat **__ENV**, jangan di-hardcode di script yang ikut ke repo.

Cara Membaca Output k6

Metrik	Artinya	Yang dilihat
http_req_duration	Waktu total satu request	p(95), p(90), med
http_req_waiting	Waktu server “mikir” (TTFB)	Tinggi = masalah di server
http_req_failed	Persentase request gagal	Sedekat mungkin ke 0%
http_reqs	Total request dan per detik	Throughput (/s)
checks	Persentase check yang lolos	Idealnya 100%
iterations	Berapa kali skenario selesai	Total dan per detik
vus_max	VU terbanyak yang aktif	Sesuai rencana?

Tanda ✓ atau ✗ di samping threshold menunjukkan mana yang lulus dan mana yang gagal.

05

Dari Hasil Test ke Perbaikan

Angka merah sudah ketemu. Terus diapain?

05 · ANALISIS

Pola yang Muncul Saat VU Terus Naik



SEBELUM JENUH

VU naik, throughput ikut naik, response time stabil.

TITIK JENUH

Throughput mentok. Ada resource yang sudah penuh.

SETELAHNYA

Request mengantre, response time melonjak, error mulai muncul.

05 · ANALISIS

Gejala, Dugaan Penyebab, Perbaikan

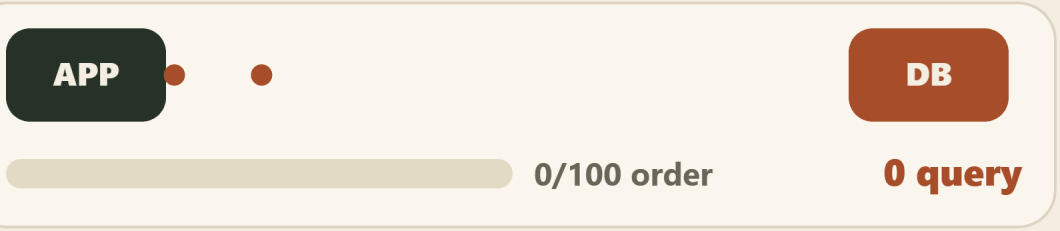
Gejala	Dugaan penyebab	Coba perbaiki dengan
Endpoint list makin lambat saat data bertambah	Query N+1, tanpa pagination	Eager loading, pagination
Pencarian atau filter lambat	Kolom tidak di-index	Tambah index di kolom yang difilter
Error “too many connections” saat VU naik	Connection pool database habis	Atur ukuran pool, pastikan koneksi ditutup
CPU 100% di satu endpoint	Proses berat di dalam request	Pindahkan ke queue atau background job
Data yang sama diminta berulang-ulang	Tidak ada cache	Cache (Redis atau in-memory)

05 · CONTOH KASUS

Tersangka Utama: Query N+1

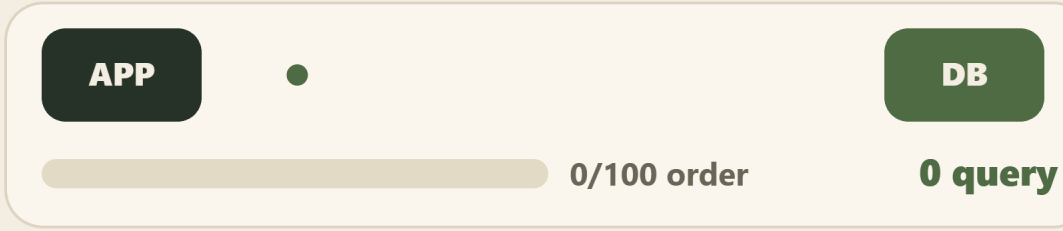
SEBELUM · 1 + N QUERY

```php
$orders = Order::all();
foreach ($orders as $order) {
  // query baru tiap order
  echo $order->user->name;
}
```



SESUDAH · EAGER LOADING

```php
$orders = Order::with('user')->get();
foreach ($orders as $order) {
  // user sudah ikut dimuat
  echo $order->user->name;
}
```



Dengan 1 user nggak kerasa. Dikali 50 VU, bedanya **5.050 vs 100 query** per putaran. Konsep sama di Prisma (*include*), EF Core (*Include*), dan JPA (*fetch join*).

05 · ANALISIS

Best Practice

1

Test di environment yang mirip production, idealnya staging.

2

Jalankan k6 dari mesin yang beda dengan server, kalau bisa.

3

Mulai dari smoke, naik pelan. Jangan langsung 1.000 VU.

4

Pakai jumlah data yang realistis, bukan tabel berisi 3 baris.

5

Ulangi 2–3 kali dan bandingkan dengan baseline sebelumnya.

6

Pantau CPU, RAM, dan database selama test berjalan.

05 · MONITORING

k6 Bilang “Lambat”, Monitoring Bilang “Kenapa”

DARI LUAR · K6

Apa yang dirasakan user

Response time, error rate, throughput.

DARI DALAM · PROMETHEUS + GRAFANA

Apa yang terjadi di server

CPU, RAM, query lambat, log error.

Meme: kucing detektif pakai kaca pembesar

Jalankan keduanya bersamaan, lalu cocokkan **jam** saat response time naik dengan grafik server di jam yang sama.



06

Hands-on Lab k6

EXPERIMENT TIME! Saatnya aplikasi kalian sendiri yang dites.

06 · HANDS-ON LAB

Tiga Level dalam Satu Jam

10 MENIT

Level 1 · Smoke

Install k6, script pertama, satu endpoint.

20 MENIT

Level 2 · Load

Login, skenario 2 endpoint, stages, threshold, dashboard.

15 MENIT

Level 3 · Stress

Stress atau spike. Cari titik jenuh aplikasi kalian.

5 + 10 MENIT

Laporan

Isi template, lalu presentasi 1 menit per tim.

Kerjakan per tim. Satu tim, satu aplikasi, satu target server. Yang sudah selesai Level 1 boleh langsung lanjut.

06 · HANDS-ON LAB

Aturan Main Sebelum Menembak

- Target **hanya** aplikasi tim sendiri: localhost atau server staging milik tim.
- Jangan ke production, server kampus, atau API pihak ketiga (payment gateway, maps, dan sejenisnya).
- Mulai kecil, naikkan pelan-pelan. Kalau error terus-terusan, tekan **Ctrl+C** dan catat angkanya.
- Pakai akun dan data test. Credential lewat **-e** atau **__ENV**, jangan di-commit.



Install k6

WINDOWS

```
winget install k6 --source winget
# atau: choco install k6
```

MACOS

```
brew install k6
```

UBUNTU (BINARY DARI GITHUB RELEASE)

```
curl -s https://api.github.com/repos/grafana/k6/releases/latest \
  | grep "browser_download_url.*linux-amd64.tar.gz" | cut -d '"' -f 4 | wget -i -
tar -xzf k6-*-linux-amd64.tar.gz && sudo mv k6-*-linux-amd64/k6 /usr/local/bin/
```

DOCKER

```
docker run --rm -i grafana/k6 run - <smoke.js
```

CEK BERHASIL

```
k6 version
```

06 · LAB LEVEL 1 · 10 MENIT

Level 1: Smoke Test

1. Jalankan aplikasi tim kalian (localhost atau staging tim).
2. Pilih **satu endpoint GET** yang tidak butuh login, misalnya list produk.
3. Buat file **smoke.js** dari slide berikut, ganti path endpoint-nya.
4. Jalankan, lalu catat p(95), error rate, dan apakah semua threshold ✓.

```
k6 run -e BASE_URL=http://localhost:3000 smoke.js
```

smoke.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 2,
  duration: '1m',
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<800'],
  },
};

export default function () {
  const res = http.get(`${__ENV.BASE_URL}/api/[endpoint-kalian]`);
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}
```

Ganti **[endpoint-kalian]** dengan path API tim.

Threshold smoke sengaja longgar (800 ms). Tujuannya memastikan semuanya jalan, bukan mengukur batas.

06 · LAB LEVEL 2 · 20 MENIT

Level 2: Load Test + Threshold

1. Tentukan “traffic normal” aplikasi kalian. Contoh: 20 VU = satu kelas pakai bareng.
2. Buat **load.js**: skenario 2 endpoint (list lalu detail) dengan think time.
3. Endpoint butuh login? Tambahkan **setup()** dari slide “Endpoint yang Butuh Login”.
4. Jalankan dengan web dashboard dan export **laporan-load.html**.
5. Lulus semua threshold? Kalau gagal, cari endpoint mana yang paling lambat.

06 · LAB LEVEL 2 · 20 MENIT

load.js

```
const BASE = __ENV.BASE_URL;
export const options = {
  stages: [
    { duration: '1m', target: 20 },
    { duration: '3m', target: 20 },
    { duration: '1m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.01'],
    checks: ['rate>0.99'],
  },
};
```

```
export default function () {
  const list = http.get(
    `${BASE}/api/[resource]`);
  check(list, {
    'list 200': (r) => r.status === 200 });
  sleep(1);
  const detail = http.get(
    `${BASE}/api/[resource]/1`);
  check(detail, {
    'detail 200': (r) => r.status === 200 });
  sleep(2);
}
```

```
k6 run load.js
```

06 · LAB LEVEL 3 · 15 MENIT

Level 3: Cari Titik Jenuh

1. Copy load.js jadi **stress.js** atau **spike.js**. Pilih salah satu.
2. Cukup ganti **stages**, dan tambahkan threshold pengaman **abortOnFail**.
3. Pantau dashboard k6 **dan** resource server (Task Manager, htop, atau Grafana).
4. Catat di VU berapa response time mulai naik tajam dan error mulai muncul.
5. ☆ **Bonus:** perbaiki satu bottleneck (index, eager loading, cache), lalu ulangi test dan bandingkan.

06 · LAB LEVEL 3 · 15 MENIT

stress.js dan spike.js

STRESS · NAIK BERTAHAP

```
stages: [
  { duration: '1m', target: 20 },
  { duration: '2m', target: 50 },
  { duration: '2m', target: 100 },
  { duration: '1m', target: 0 },
],
```

SPIKE · MELONJAK MENDADAK

```
stages: [
  { duration: '30s', target: 5 },
  { duration: '10s', target: 100 },
  { duration: '1m', target: 100 },
  { duration: '10s', target: 5 },
  { duration: '30s', target: 0 },
],
```

PENGAMAN · TEST BERHENTI SENDIRI KALAU ERROR DI ATAS 5%

```
http_req_failed: [{ threshold: 'rate<0.05', abortOnFail: true }],
```

06 · LAB · CONTOH LAPORAN (DATA ILUSTRASI)

Contoh Laporan: Tim Toko Buku

Item	Hasil Tim Toko Buku
Endpoint yang diuji	GET /api/products dan GET /api/products/:id
Jenis test dan stages	Load 20 VU selama 5 menit, lalu Stress naik sampai 100 VU
VU maksimal	100 VU
p(95) response time	420 ms di 20 VU (lolos) · 2,3 detik di 100 VU (gagal)
Error rate	0% di 20 VU · 6,8% di 100 VU
Throughput (req/s)	±12 req/s di 20 VU · mentok di ±38 req/s
Titik jenuh	±60 VU: p95 melonjak, CPU database 95%
Dugaan bottleneck dan rencana perbaikan	Query N+1 di list produk (relasi kategori). Rencana: eager loading + pagination, lalu test ulang.

KESIMPULAN

Aman untuk satu kelas (20 user). Belum siap kalau dipakai serentak lebih dari 60 user.

KETERBATASAN

k6 dan server jalan di laptop yang sama. Data produk baru 500 baris.

Template Laporan Hasil Test

Item	Isi tim kalian
Endpoint yang diuji	[ ]
Jenis test dan stages	[ ]
VU maksimal	[ ]
p(95) response time	[ ]
Error rate	[ ]
Throughput (req/s)	[ ]
Titik jenuh (di VU berapa)	[ ]
Dugaan bottleneck dan rencana perbaikan	[ ]

LAMPIRKAN

laporan-load.html dari web dashboard dan script k6 kalian.

PRESENTASI

1 menit per tim: angka paling mengejutkan dan dugaan penyebabnya.

PENUTUP

Rekap Sesi Hari Ini

1. Lolos functional test belum berarti kuat dipakai banyak orang. Performa juga fitur.

2. Baca persentil (p95, p99) dan error rate. Rata-rata bisa menipu.

3. Pilih test dari pertanyaannya: smoke, load, stress, spike, soak, breakpoint.

4. k6 + threshold menjadikan performa gerbang otomatis di pipeline. Dan hanya tes sistem milik sendiri.

SHARING KNOWLEDGE

Terima Kasih

